

Dziedziczenie zaawansowane i interfejsy w Pythonie — czas: 120 minut

Część A — Dziedziczenie klasyczne: nadpisywanie i super() (25 min)

Zadanie 1 — Nadpisywanie metod

Utwórz klasę Employee, która:

- posiada pole name,
- posiada metodę get_salary(), która zwraca 3000.

Utwórz klasę Manager(Employee), która:

- nadpisuje metodę get_salary() tak, aby zwracała 6000.

Utwórz obiekty obu klas i wypisz ich wynagrodzenia.

Zadanie 2 — Konstruktor i super()

Rozszerz klasy z Zadania 1:

1. Dodaj do Employee pole department.
 2. Zaimplementuj konstruktor w Employee.
 3. W klasie Manager użyj super().__init__().
 4. Dodaj pole bonus tylko dla Manager.
 5. Zmień get_salary() tak, aby uwzględniała bonus.
-

Zadanie 3 — Metody wspólne i różne

Utwórz klasy:

- Vehicle – metoda move() → wypisuje „Pojazd się porusza”,
- Car(Vehicle) – nadpisuje move(),
- Bike(Vehicle) – nadpisuje move().

Utwórz listę pojazdów i wywołaj move() dla każdego obiektu.

Część B — Wielopoziomowe dziedziczenie i MRO (20 min)

Zadanie 4 — Dziedziczenie wielopoziomowe

Utwórz klasy:

- Person,
- Employee(Person),
- Programmer(Employee).

Każda klasa dodaje jedno własne pole.

Dodaj metodę info(), która wykorzystuje dane ze wszystkich poziomów.

Zadanie 5 — Wielokrotne dziedziczenie

Utwórz klasy:

- Flyer z metodą fly(),
- Swimmer z metodą swim(),
- Duck(Flyer, Swimmer).

Utwórz obiekt klasy Duck i wywołaj obie metody.

Wyświetl kolejność MRO:

```
print(Duck.mro())
```

Część C — Interfejsy w Pythonie (ABC, abstractmethod) (25 min)

Zadanie 6 — Klasa abstrakcyjna jako interfejs

Używając ABC i @abstractmethod, utwórz interfejs Shape, który:

- wymusza metodę area().

Utwórz klasy:

- Rectangle(Shape),
- Circle(Shape).

Każda klasa ma poprawnie implementować area().

Przetestuj w pętli listę figur.

Zadanie 7 — Próba utworzenia instancji klasy abstrakcyjnej

Spróbuj utworzyć obiekt klasy Shape.

Zaobserwuj i opisz błąd.

Wyjaśnij, dlaczego Python nie pozwala na taką operację.

Zadanie 8 — Interfejs jako Protocol (duck typing)

Użyj typing.Protocol, aby utworzyć interfejs Drawable z metodą:

```
def draw(self) -> None
```

Utwórz klasy:

- Window,
- Image.

Obie muszą implementować metodę draw().

Napisz funkcję render(obj: Drawable) i przetestuj dla obu klas.

Część D — Kompozycja zamiast dziedziczenia (25 min)

Zadanie 9 — Błędne użycie dziedziczenia

Utwórz klasy:

- Engine,
- Car(Engine).

Zaimplementuj prostą metodę uruchamiania.

Następnie:

- oceń, dlaczego to jest **błędny model dziedziczenia**,
 - zapisz w komentarzu, jaka relacja semantyczna została naruszona.
-

Zadanie 10 — Poprawa przez kompozycję

Popraw Zadanie 9 tak, aby:

- Car posiadał pole engine,
 - metoda start() delegowała wywołanie do obiektu Engine.
-

Zadanie 11 — Kompozycja z wymiennym komponentem

Utwórz klasy:

- FileLogger,
- DatabaseLogger.

Utwórz klasę Application, która:

- otrzymuje logger w konstruktorze,
- posiada metodę run() wywołującą logger.log().

Przetestuj program z dwoma różnymi loggerami.

Część E — Mini-projekt: system płatności (20–25 min)

Zadanie 12 — Interfejs płatności

Utwórz interfejs PaymentMethod (klasa abstrakcyjna), który wymusza metodę:

```
def pay(self, amount: float)
```

Utwórz klasy:

- CardPayment,
- BlikPayment,
- CashPayment.

Każda klasa ma inaczej realizować metodę pay().

Zadanie 13 — Kompozycja w systemie sklepowym

Utwórz klasę Order, która:

- posiada pole payment_method,
- posiada metodę process_payment(amount) wywołującą pay().

Przetestuj działanie z różnymi metodami płatności.

Zadanie 14 — Dziedziczenie vs kompozycja — porównanie

W komentarzu do kodu zapisz:

- który fragment wykorzystuje dziedziczenie,
 - który wykorzystuje kompozycję,
 - dlaczego w systemie płatności **kompozycja jest lepszym wyborem**.
-

Zadania dodatkowe (dla chętnych)

Zadanie 15 — LSP (zasada podstawienia Liskov)

Zmodyfikuj jedną klasę płatności tak, aby **łamiała zasadę LSP**.
Sprawdź, jaki wpływ ma to na działanie klasy Order.

Zadanie 16 — Refaktoryzacja dziedziczenia na kompozycję

Weź jedno z wcześniejszych zadań opartych na dziedziczeniu i przepis je w całości z użyciem kompozycji.